

AUTOJUDGE

Introduction and Problem Statement :

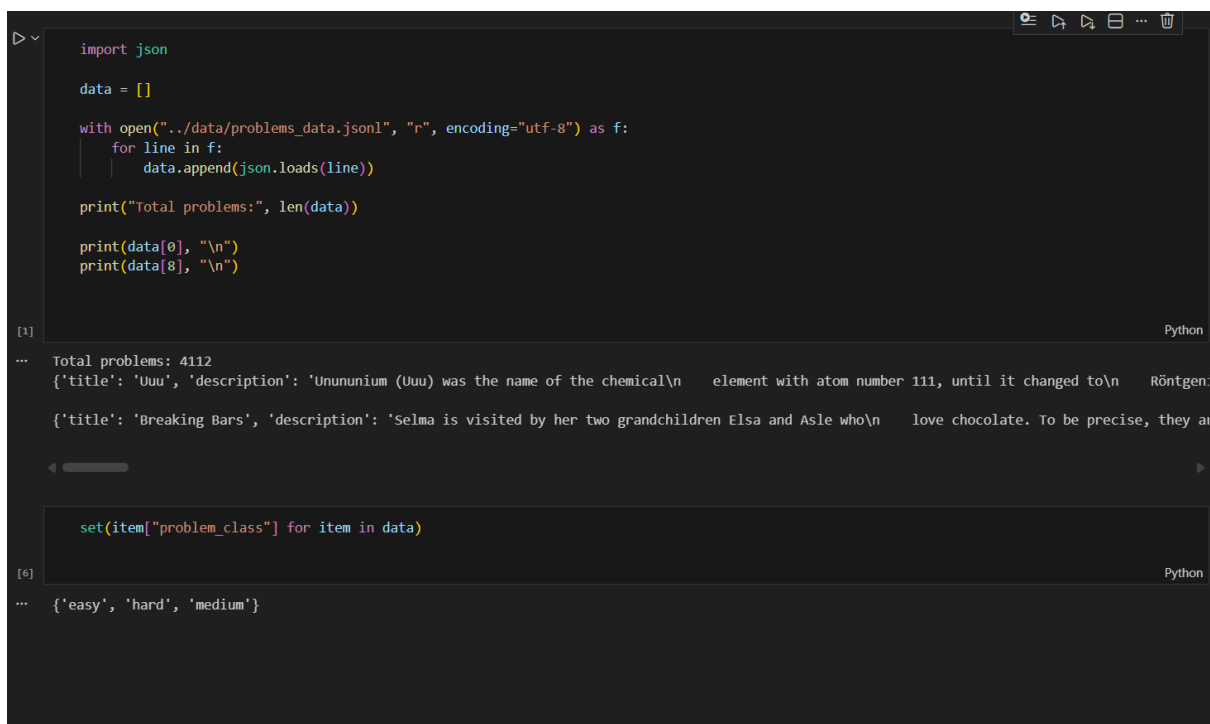
With the rapid growth of competitive programming platforms and online coding resources, a large number of programming problems are made available to learners and professionals. These problems are usually categorized into difficulty levels such as *Easy*, *Medium*, and *Hard*. However, this categorization is often performed manually and can be highly subjective, varying across platforms and problem setters.

Accurate estimation of problem difficulty is important for multiple reasons. Beginners benefit from problems suited to their skill level, educators require structured problem sets, and platforms aim to recommend appropriate challenges to users. Manual labeling of difficulty is time-consuming and inconsistent, motivating the need for automated solutions.

The goal of this project, **AutoJudge**, is to predict the difficulty of programming problems using **only the textual description of the problem**. The system predicts:

- A **difficulty class**: Easy, Medium, or Hard
- A **numerical difficulty score** for finer-grained estimation

This problem is challenging because difficulty is subjective and depends on the intended algorithm, which is often not explicitly mentioned in the problem text. AutoJudge addresses this challenge using machine learning models trained on previously labeled problems.



```
import json

data = []

with open("../data/problems_data.jsonl", "r", encoding="utf-8") as f:
    for line in f:
        data.append(json.loads(line))

print("Total problems:", len(data))

print(data[0], "\n")
print(data[8], "\n")

[1] Python

... Total problems: 4112
{'title': 'Uuu', 'description': 'Ununium (Uuu) was the name of the chemical\n    element with atom number 111, until it changed to\n    Röntgen.

{'title': 'Breaking Bars', 'description': 'Selma is visited by her two grandchildren Elsa and Asle who\n    love chocolate. To be precise, they a

[6] Python

... {'easy', 'hard', 'medium'}
```

Dataset Description

- Dataset Overview

The dataset used in this project consists of programming problems collected from competitive programming platforms. The dataset is stored in JSON Lines format: `data/problems_data.jsonl`

Each line in the file corresponds to a single programming problem.

Each problem entry contains the following fields:

Field Name	Description
Title	Title of the problem
Description	Main problem statement
Input description	Input format specification
Output description	Output format specification
Problem class	Difficulty label (Easy / Medium / Hard)
Problem score	Numerical difficulty score

- Dataset Characteristics

- The dataset includes problems of varying length and complexity.
- Difficulty labels are **subjective** and may differ across platforms.
- Some classes (especially Medium) are more frequent, leading to class imbalance.
- Numerical difficulty scores provide a smoother target compared to class labels.

These characteristics reflect real-world data and influence the achievable performance of machine learning models.

Data Preprocessing and Feature Engineering

- Text Preprocessing

Before feature extraction, the problem text undergoes preprocessing to ensure consistency and reduce noise. The following steps are applied:

- Conversion of text to lowercase
- Removal of extra whitespace
- Merging of problem description, input format, and output format into a single text block

This ensures that all relevant information is captured in one textual input.

Text Features

Textual data is transformed into numerical form using **TF-IDF (Term Frequency–Inverse Document Frequency)**. TF-IDF represents each problem as a high-dimensional sparse vector, capturing the importance of words across the dataset.

Key aspects:

- Unigrams and bigrams are used
- Stop words are removed
- The resulting vectors are suitable for linear models

Numeric Features (Regression Only)

For predicting numerical difficulty scores, simple numeric features are added:

- Length of the problem text
- Word count

These features provide additional signals related to problem complexity.

Pipeline Summary:

Raw Problem Text



Text Preprocessing



TF-IDF Vectorization



ML Models

Models and Experimental Setup

Classification Model

The difficulty classification task is handled using a **Linear Support Vector Classifier (LinearSVC)**. This model is well-suited for high-dimensional sparse text data produced by TF-IDF.

Reasons for choosing LinearSVC:

- Efficient for large feature spaces
- Strong performance on text classification tasks
- Robust to overfitting with proper regularization

The classifier predicts one of three classes: Easy, Medium, or Hard.

Regression Model

To predict a continuous difficulty score, a **Random Forest Regressor** is used. This ensemble-based model captures non-linear relationships and works well with mixed feature types.

Reasons for choosing Random Forest:

- Handles non-linear patterns
- Robust to noise
- Provides stable predictions for continuous targets

Experimental Setup

- Dataset split: 80% training, 20% testing
- Models are trained offline
- A single fitted TF-IDF vectorizer is reused for both training and inference
- The web application performs inference only and does not retrain models

Libraries used:

- scikit-learn
- numpy
- scipy
- flask

Results

Classification Results

The classification model is evaluated using **accuracy** on the test set.

- Random baseline accuracy (3 classes): **33%**
- Achieved accuracy: **0.5006**

Due to the subjective nature of difficulty labels and text-only input, perfect accuracy is not achievable. The model often confuses Medium with Easy or Hard, which is expected.

Confusion Matrix

A confusion matrix is used to analyze misclassifications across classes.

Observations:

- Medium problems are the most difficult to classify
- Easy and Hard classes are better separated

```
[Running] python -u "c:\Users\hp\OneDrive\Desktop\Auto Judge\src\train_classifier.py"
Building text features...
Loading labels...
Splitting dataset...
Training classifier...
Evaluating model...

Accuracy: 0.5006
```

		precision	recall	f1-score	support
	0	0.48	0.43	0.45	153
	1	0.37	0.30	0.33	281
	2	0.57	0.67	0.62	389
	accuracy			0.50	823
	macro avg	0.47	0.47	0.47	823
	weighted avg	0.49	0.50	0.49	823

```
Classifier saved at: C:\Users\hp\OneDrive\Desktop\Auto Judge\models\classifier.pkl
```

Regression Results

Regression performance is evaluated using:

- **MAE: 1.68**
- **RMSE: 2.03**

Results indicate that the predicted difficulty score deviates by a small margin on average, making it useful for fine-grained difficulty estimation.

```
[Running] python -u "c:\Users\hp\OneDrive\Desktop\Auto Judge\src\train_regressor.py"
Building regression features...
Loading targets...
Splitting dataset...
Training regressor...
Evaluating model...
MAE : 1.6714
RMSE : 2.0223
Regressor saved at: C:\Users\hp\OneDrive\Desktop\Auto Judge\models\regressor.pkl
```

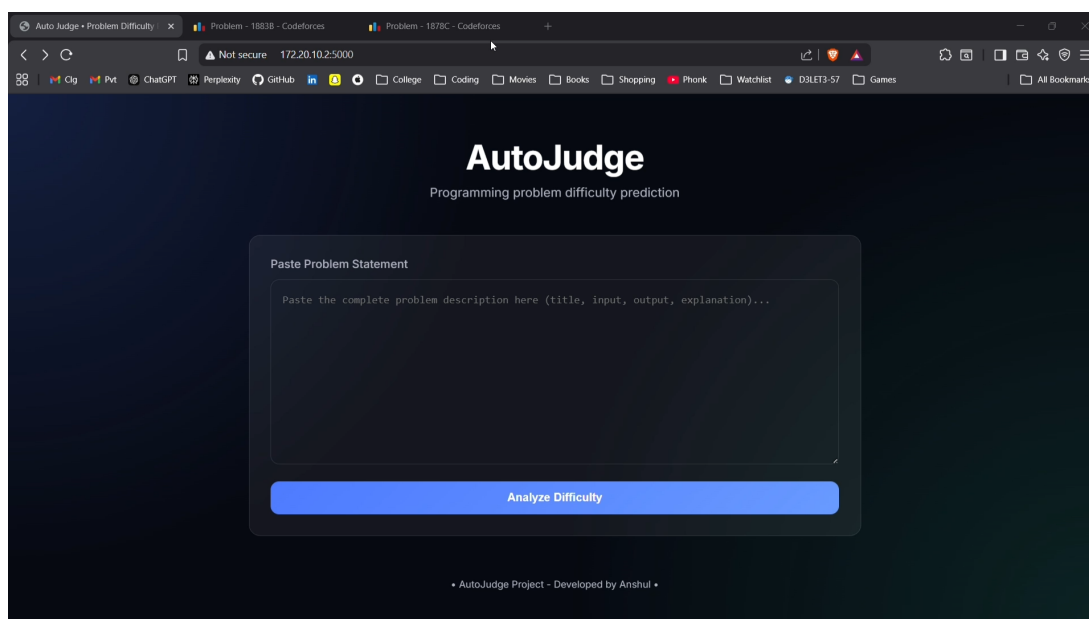
Web Interface and Conclusion

Web Interface Description

AutoJudge includes a Flask-based web interface that allows users to interact with the trained models.

Features:

- Text box to paste a problem statement
- Button to trigger prediction
- Display of difficulty class and numerical score
- Warnings for short or low-confidence inputs



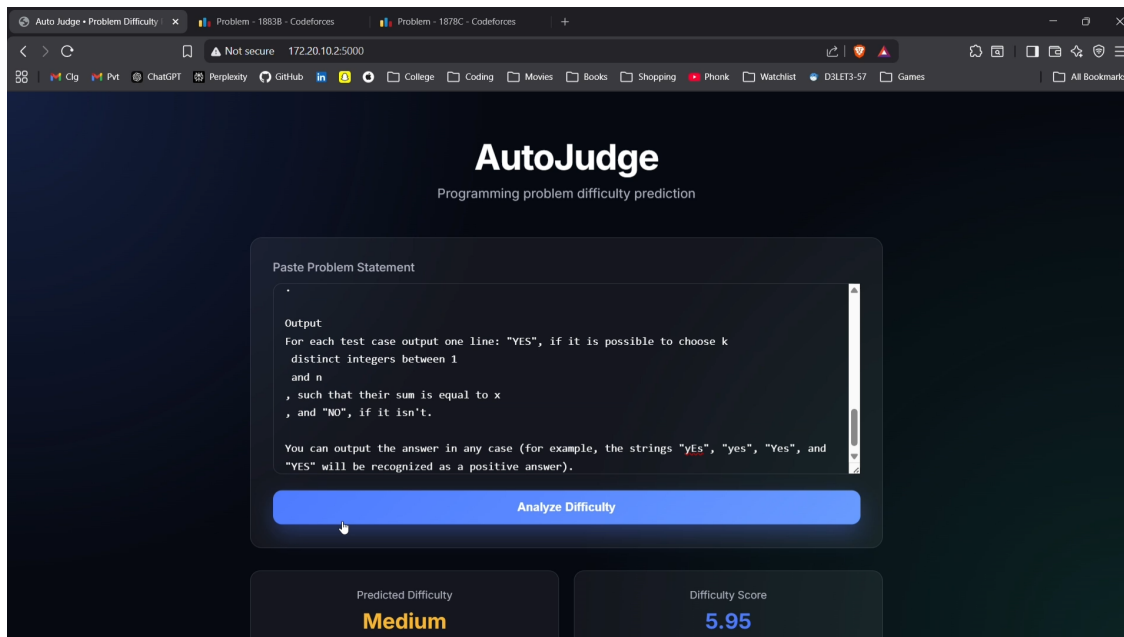


Figure shows the AutoJudge web interface along with a sample difficulty prediction.

Conclusion

This project presented **AutoJudge**, a machine learning system for predicting the difficulty of programming problems using textual descriptions. The system integrates preprocessing, feature engineering, classification, regression, and deployment into a single pipeline.

While difficulty prediction remains a subjective and challenging task, AutoJudge demonstrates that meaningful predictions can be achieved using machine learning. Future work may include incorporating solution code, platform metadata, or advanced language models to improve accuracy.